

Evaluación Final Transversal

Ejecución práctica

Contexto

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
FPY1101	Fundamentos de programación	4 horas	40%

1.Situación Evaluativa 1:

X	Ejecución práctica
---	--------------------

2.Instrucciones Evaluación Final Transversal

La empresa de transporte **RutaExpress** requiere un programa en Python para administrar sus recorridos de bus y la disponibilidad de asientos por servicio. Todo el comportamiento del sistema debe organizarse en funciones bien definidas. El programa incluye un menú interactivo, validaciones de entrada y una separación clara entre la lógica de cada función y las decisiones del programa principal.

1. Datos que debe manejar el sistema

El sistema trabaja con **dos diccionarios relacionados**, ambos identificados por el mismo código de recorrido como clave. Estos diccionarios deben existir desde que el programa inicia y permanecer disponibles durante toda la ejecución.

Estos diccionarios deben crearse en el programa principal y pasarse como argumento a todas las funciones que necesiten leerlos o modificarlos (aunque estas no los soliciten en el enunciado). No está permitido acceder a ellos como variables globales dentro de las funciones. También es importante indicar que cada estructura de repetición (while, for) debe utilizarse según su propósito natural (dependiendo de si la condición de parada cambia durante la ejecución o si el volumen de iteraciones se conoce de antemano). Usar una estructura fuera de ese contexto será considerado un error de diseño y no cumplirá con el estándar de la evaluación.

Diccionario recorridos

Contiene la información descriptiva de cada recorrido. La **clave** es el código de recorrido y el **valor** es una lista con los siguientes campos, en este orden:

Campo	Qué representa	Restricciones de validación
"origen"	Ciudad de origen del recorrido	No debe contener solo espacios en blanco ni estar vacío
"destino"	Ciudad de destino del recorrido	No debe contener solo espacios en blanco ni estar vacío
"distancia_km"	Distancia total del recorrido en kilómetros	Número entero mayor que cero
"tipo_bus"	Tipo de bus asignado al servicio	Debe ser exactamente 'normal', 'semi-cama' o 'cama'
"servicio"	Turno del servicio	Debe ser exactamente 'día' o 'noche'
"tiene_wifi"	Indica si el bus cuenta con WiFi a bordo	El usuario ingresa 's' o 'n'. El sistema lo convierte a True o False

Los puntos suspensivos en el siguiente ejemplo indican que pueden existir más registros:

```
recorridos = {  
    'R001': ['Santiago', 'Valparaíso', 120, 'normal', 'día', True],  
    'R002': ['Santiago', 'Concepción', 500, 'cama', 'noche', True],  
    'R003': ['La Serena', 'Coquimbo', 15, 'normal', 'día', False],  
    'R004': ['Temuco', 'Valdivia', 165, 'semi-cama', 'día', True],  
    'R005': ['Iquique', 'Arica', 310, 'cama', 'noche', False],  
    'R006': ['Santiago', 'Rancagua', 90, 'normal', 'día', True],  
    ...  
}
```

Diccionario venta

Contiene la información operativa de cada recorrido. La **clave** es el mismo código de recorrido y el **valor** es una lista con los siguientes dos campos:

Campo	Qué representa	Restricciones de validación
"precio"	Precio del pasaje en pesos	Número entero mayor que cero

"asientos"	Cantidad de asientos disponibles para ese servicio	Número entero mayor o igual a cero
------------	--	------------------------------------

Los puntos suspensivos en el siguiente ejemplo indican que pueden existir más registros:

```
venta = {  
    'R001': [7990, 20],  
    'R002': [25990, 0],  
    'R003': [1990, 35],  
    'R004': [12990, 8],  
    'R005': [18990, 3],  
    'R006': [4990, 12],  
    ...  
}
```

2. Lo que debe hacer el sistema

El sistema se controla desde un menú que aparece en pantalla cada vez que el usuario termina una acción. El usuario elige una opción numérica, el programa ejecuta la tarea correspondiente y vuelve a mostrar el menú. Esto se repite hasta que el usuario elige salir. Si el usuario ingresa un valor que no corresponda a ninguna opción válida, el sistema muestra el mensaje "Debe seleccionar una opción válida" y vuelve a mostrar el menú.

Para la lectura de la opción del menú:

Define una función llamada `leer_opcion()`. No recibe parámetros. Solicita al usuario que ingrese una opción, valida que el valor ingresado sea un número entero y que esté dentro del rango de opciones válidas del menú, y retorna ese valor entero. Si el usuario ingresa un dato que no es un entero debe manejarlo mediante excepciones.

```
===== MENÚ PRINCIPAL =====  
1. Asientos por ciudad de origen  
2. Búsqueda de recorridos por rango de precio  
3. Actualizar precio de recorrido  
4. Agregar recorrido  
5. Eliminar recorrido  
6. Salir  
=====
```

A continuación, se describe qué debe ocurrir al elegir cada opción:

Opción 1 — Asientos por ciudad de origen

El sistema solicita al usuario el nombre de una ciudad de origen (por ejemplo: Santiago, Temuco o La Serena). La búsqueda **no distingue entre mayúsculas y minúsculas**, por lo que "Santiago" y "SANTIAGO" deben producir el mismo resultado. El sistema recorre el diccionario **recorridos** identificando todos los recorridos que partan desde esa ciudad. Por cada recorrido encontrado, se debe buscar su código en el diccionario **venta**, extraer la cantidad de asientos disponibles (el segundo elemento de la lista) y acumularla en un total. Una vez procesados todos los recorridos, se debe mostrar dicho total acumulado en pantalla.

Para implementar esta opción:

Define una función llamada **asientos_origen(origen)**. Recibe la ciudad de origen como parámetro, **no retorna ningún valor** y muestra el resultado directamente por pantalla.

Opción 2 — Búsqueda de recorridos por rango de precio

El sistema solicita al usuario un precio mínimo y un precio máximo. Luego recorre el diccionario **venta** y construye una lista con todos los recorridos que: (a) tengan un precio dentro del rango ingresado, y (b) tengan asientos disponibles (asientos distintos de cero). Cada elemento de la lista tiene el formato "Origen-Destino--Código". Los resultados se muestran ordenados **alfabéticamente**. Si no hay recorridos que cumplan las condiciones, el sistema muestra: "No hay recorridos en ese rango de precios."

Restricciones de entrada:

El precio mínimo y máximo deben ingresarse como valores enteros. Esta validación ocurre en el **programa principal**, antes de llamar a la función. Como el usuario puede ingresar cualquier tipo de dato, debe utilizarse manejo de excepciones. Si el dato ingresado no es un entero válido, el sistema muestra "*Debe ingresar valores enteros*" y vuelve a solicitar ambos valores.

Para implementar esta opción:

Define una función llamada **busqueda_precio(p_min, p_max)**. Recibe el precio mínimo y máximo como parámetros (*estos valores deben ser mayores o iguales a cero y el p_min menor o igual al p_max*), **no retorna ningún valor** y muestra los resultados directamente por pantalla.

Opción 3 — Actualizar precio de recorrido

El sistema solicita al usuario el código del recorrido y el nuevo precio que se desea asignar. Si el código existe en el diccionario **venta**, el sistema actualiza su precio. Si el código no existe, informa al usuario. Al terminar, pregunta: "*¿Desea actualizar otro precio (s/n)?*": si la respuesta es "s", el proceso se repite; si es "n", el programa vuelve al menú principal.

Para implementar esta opción:

Define una función **buscar_codigo(codigo)** que recorra el diccionario y retorne True si el código existe, o False si no existe.

Define una función **actualizar_precio(codigo, nuevo_precio)** que debe:

- Verificar la existencia del código (se recomienda invocar a `buscar_codigo` internamente para evitar duplicar la lógica de búsqueda).
- Si el código existe, actualizar el precio en el diccionario y retornar True.
- Si el código no existe, retornar False.

El **programa principal** debe llamar a `actualizar_precio` y, basándose exclusivamente en el valor booleano retornado, decidir qué imprimir: "Precio actualizado" si fue True, o "El código no existe" si fue False.

Recordar que el `nuevo_precio` debe ser un valor entero positivo y la validación del código no debe distinguir mayúsculas y minúsculas.

Opción 4 — Agregar recorrido

El sistema solicita al usuario todos los datos del nuevo recorrido: código, origen, destino, distancia, tipo de bus, servicio, si tiene WiFi, precio y asientos. Antes de crear el registro, **cada dato es validado de forma independiente**. Si algún dato no cumple su condición, el sistema informa al usuario y **no registra el recorrido**. Solo cuando todos los datos son válidos y el código no existe previamente, el sistema agrega el registro en ambos diccionarios.

La siguiente tabla resume las condiciones que debe cumplir cada campo:

Campo solicitado	Condición de validación
código	No vacío ni solo espacios en blanco, y que no exista ya en los diccionarios
título	No vacío ni solo espacios en blanco
plataforma	No vacía ni solo espacios en blanco
género	No vacío ni solo espacios en blanco
código	No vacío ni solo espacios en blanco, y que no exista ya en los diccionarios
origen	No vacío ni solo espacios en blanco
destino	No vacío ni solo espacios en blanco
distancia	Número entero mayor que cero
tipo_bus	Debe ser exactamente 'normal', 'semi-cama' o 'cama'

Para implementar esta opción:

Define una función de validación independiente para cada campo de la tabla anterior. Cada función recibe únicamente el dato a validar, aplica su condición y retorna **True** si es válido o **False** si no lo es. Los mensajes de error **no** se muestran dentro de las funciones de validación.

En el **programa principal**, al elegir esta opción, se solicitan los datos al usuario y se llama a cada función de validación. Si alguna retorna **False**, el programa muestra el mensaje de error correspondiente y no registra el recorrido.

Solo si todas las validaciones retornan **True**, el programa llama a la función **agregar_recorrido(codigo, origen, destino, distancia, tipo_bus, servicio, tiene_wifi, precio, asientos)**, que agrega el registro en ambos diccionarios y retorna **True**. Si el código ya existía, retorna **False**. El programa principal muestra: "Recorrido agregado" o "El código ya existe" según corresponda.

Opción 5 — Eliminar recorrido

El sistema solicita el código del recorrido que se desea eliminar. Si el código existe, elimina el registro en **ambos diccionarios** (recorridos y venta) e informa que la operación fue exitosa. Si el código no existe, informa al usuario.

Para implementar esta opción:

Reutiliza la función **buscar_codigo(codigo)** definida anteriormente para verificar la existencia del recorrido.

Define una función **eliminar_recorrido(codigo)** que debe:

- Verificar la existencia del código (se recomienda invocar a **buscar_codigo** internamente para evitar duplicar la lógica de búsqueda).
- Si el código existe, eliminar el registro en ambos diccionarios y retornar **True**.
- Si el código no existe, retornar **False**.

El **programa principal** debe llamar a **eliminar_recorrido** y, basándose exclusivamente en el valor booleano retornado, decidir qué imprimir: "Recorrido eliminado" si fue **True**, o "El código no existe" si fue **False**.

Recordar que la validación del código no debe distinguir mayúsculas y minúsculas.

Opción 6 — Salir

El sistema termina la ejecución de forma limpia. El ciclo del menú se detiene y el programa finaliza mostrando el mensaje: *"Programa finalizado."*

Para implementar esta opción:

Esta opción no requiere función adicional. El programa principal es responsable de detener el ciclo del menú y mostrar el mensaje de cierre.

3. Ejemplo de ejecución

A continuación, se muestra un ejemplo representativo de cómo debe funcionar el programa. Los datos en **negrita** son valores ingresados por el usuario:

```
===== MENÚ PRINCIPAL =====
1. Asientos por ciudad de origen
2. Búsqueda de recorridos por rango de precio
3. Actualizar precio de recorrido
4. Agregar recorrido
5. Eliminar recorrido
6. Salir
=====

Ingrese opción: 1
Ingrese ciudad de origen a consultar: SANTIAGO
El total de asientos disponibles es: 32

Ingrese opción: 2
Ingrese precio mínimo: hola
Debe ingresar valores enteros
Ingrese precio mínimo: 1000
Ingrese precio máximo: 10000
```

Los recorridos encontrados son: ['La Serena-Coquimbo--R003', 'Santiago-Rancagua--R006', 'Santiago-Valparaíso--R001']

Ingrese opción: 3

Ingrese código del recorrido: S010

Ingrese nuevo precio: 19990

El código no existe

¿Desea actualizar otro precio (s/n)? : n

Ingrese opción: 4

Ingrese código del recorrido: R200

Ingrese origen: Santiago

Ingrese destino: Chillan

Ingrese distancia (km): 410

Ingrese tipo de bus (normal/semi-cama/cama): semi-cama

Ingrese servicio (dia/noche): noche

¿Tiene WiFi? (s/n): s

Ingrese precio: 17990

Ingrese asientos: 6

Recorrido agregado

Ingrese opción: 6

Programa finalizado.